

Assignment 1: Lexical & Semantic Retrieval on EB-NeRD and MIND

Sriharini Margapuri (2026701030)

GitHub Repo: <https://github.com/sri-harini-m-IITH-assignments/M26CS4.406-Assignment-1>

What I Built and Key Design Choices:

1. Data Pipeline: I built a reproducible, one-command pipeline that cleans and formats the raw data from both the MIND-small and EB-NeRD datasets. To strictly prevent future-click leakage, I use a time-based splitting strategy where the train, test, and validation datasets are partitioned using the timestamps. More specifically, the last day is set for the test set, the day before for validation set, and the rest for the training set. Finally, all files are in unified schemas in parquet files.

2. Lexical Retrieval (BM25): I implemented BM25 using the `rank_bm25` Python package. This package allows for the direct, lightweight, in-memory implementation of the classic Okapi BM25 algorithm developed by Stephen Robertson and Karen Spärck Jones. The query is the concatenation of the user's full reading history. BM25 (Best Matching 25) is a probabilistic ranking algorithm used to score the relevance of a document to a given user query. It is built on top of the TF-IDF, primarily to solve the "term saturation" issue that occurred in the original TF-IDF algorithm.

3. Semantic Retrieval (Embeddings + FAISS): I generated embeddings for the title and abstract for both datasets using lightweight BERT-based sentence transformers (`all-MiniLM-L6-v2` for English MIND, and `paraphrase-multilingual-MiniLM-L12-v2` for Danish EB-NeRD). After generating the title + abstract embeddings, I calculated a mean-pooled vector of the embeddings from their entire click history. For cold-start users with no history, the system defaults to a global centroid (the average of all article embeddings) to prevent failures. The index is built by taking the array of article embeddings, applying L2 normalization, and loading them into an `IndexFlatIP` (Inner Product) structure. Because all vectors are normalized to a length of 1, the fast inner product calculation mathematically equates to Cosine Similarity.

4. Evaluation: (Table of Results)

Metric	BM25 (MIND)	FAISS (MIND)	BM25 (EB-NeRD)	FAISS (EB-NeRD)
Recall@50	0.0141	0.0119	0.0191	0.0040
Recall@100	0.0238	0.0198	0.0327	0.0111
Recall@200	0.0371	0.0322	0.0546	0.0189

AUC	0.5903	0.6365	0.5239	0.5460
MRR	0.3295	0.3458	0.3371	0.3333
nDCG@5	0.3031	0.3262	0.3688	0.3722
nDCG@10	0.3605	0.3849	0.4506	0.4517
Diversity	0.9266	0.8881	0.8505	0.8282
Novelty	13.0329	13.0271	12.9597	12.9214
Coverage	0.0565	0.0536	0.0734	0.0742

Alternatives considered and why I chose what I did:

1. For BM25, I also used the 5 and 15 most recent articles to prioritize recent intent and faster evaluation. However, while evaluation time decreased, overall metrics also decreased along with it, observing that richer history captures broader signals.
2. When a user had no valid history, standard mean pooling would fail. Instead of dropping these users or scoring them randomly, I computed a global centroid (the mean of all article embeddings) as a fallback representation.
3. Given the small scale of MIND-Small and EB-NeRD Demo, I opted for IndexFlatIP (brute-force exact search) to guarantee a 100% recall of the semantic vectors at the cost of slightly higher execution time.
4. I generated embeddings for the titles and abstracts to ensure uniformity and ease of usage of said embeddings.
5. FAISS offers 100% exact retrieval accuracy with seamless python integration. I also have implemented FAISS before in earlier projects which is why I preferred it over the other ANN methods mentioned
6. Implemented an in-memory query cache for BM25 to prevent redundant indexing. This brought the total execution time from 8 hours to roughly 2 hours. This wasn't implemented in FAISS as FAISS ran in under 10 minutes normally.
7. For the large version of the dataset, my pipeline took 300+ hours. To combat this I switched from `iterrows` to `itertuples` which only calculates the scoring for the articles that matter to a particular user, lowering the time to around 30 minutes.

Observations from experiments (lexical vs. semantic, dataset differences):

1. Performance: Semantic retrieval (FAISS) consistently outperformed Lexical retrieval (BM25) on overall accuracy metrics across both datasets. For instance, on MIND, FAISS

achieved an AUC of 0.6365 compared to BM25's 0.5903. On EB-NeRD, FAISS achieved an AUC of 0.5460 compared to BM25's 0.5239.

2. Dataset Characteristics: EB-NeRD validation had an extremely small pool of cold-start users ($n=16$) compared to MIND ($n=5389$). Consequently, the confidence intervals for EB-NeRD cold-start metrics were exceptionally wide (e.g., AUC 95% CI [0.4477, 0.7106]).

3. Cold vs. Warm Users: Surprisingly, MRR and nDCG were occasionally higher for cold-start users than warm users (e.g., MIND FAISS cold MRR is 0.3365 vs. warm MRR 0.3477). This is likely because cold users are presented with fewer, highly popular candidate articles, making the ranking task artificially simpler. Predictably, coverage for cold slices was much lower (0.0284) than for warm slices (0.0500).

4. Lexical vs. Semantic Retrieval: While BM25 and FAISS yield relatively close Recall numbers at $K@ \{50, 100, 200\}$, semantic retrieval (FAISS) delivers a noticeable boost in ranking accuracy metrics (such as AUC jumping from 0.5903 to 0.6365 on MIND). This indicates that while both approaches capture a similar proportion of candidate pool items, the embedding-based vector representations rank the true positives more effectively near the top.

5. Diversity: BM25 consistently demonstrates higher list diversity (e.g., 0.9266 vs. 0.8881 on MIND) compared to semantic retrieval, as exact keyword matching pulls in a wider lexical spread, whereas dense embeddings tend to cluster around semantic centroids.

Where the Pipeline Breaks at 10x Scale:

1. Pandas Processing: The pipeline relies heavily on pandas, iterating over the Data Frame row-by-row. For the small datasets used in the assignment, this works perfectly fine, but as we increase the dataset size by 10 times (using the large versions of these datasets), this pipeline runs into Out-of-Memory errors and is immensely slow (going from 2 hours to about 300 hours in processing for BM25 and going from 2 minutes to about 30 minutes for FAISS). To prevent this, we would use `DASK` or `Polars`.

2. Creating Embeddings: Creating embeddings for datasets of this size also takes a lot of computation in both time and memory. While the models are small enough (~23M parameters) to run on a local system with a decent GPU, the size of the dataset enables more overhead.

3. Lexical Indexing: The `rank_bm25` package loads the entire term-document matrix into memory. Scaling this by 10x would require a disk-backed solution like `Elasticsearch` or `OpenSearch`.

4. Semantic Indexing: The `IndexFlatIP` method computes the inner product between the user vector and every article vector in the corpus. At a 10x larger article catalog, brute-force search latency would become a bottleneck, requiring a shift to ANN structures like `IndexIVFFlat` or `IndexHNSWFlat`.

Leaderboard screenshots:

1. MIND News Recommendation Competition

ID # ▾	File name	Date	Status	Score	Actions
896023	prediction.zip	2026-08-21 12:02	Finished	0.4936	 
895464	prediction.zip	2026-08-20 22:36	Finished	0.4915	  

52	sriharinim	2026-08-21 12:02	896023	0.4936	0.2227	0.2278	0.2828
----	------------	------------------	--------	--------	--------	--------	--------

2. Ekstra Bladet News Recommendation Competition

ID # ▾	File name	Date	Status	Score	Detailed Results	Actions
899685	predictions.zip	2026-08-24 17:18	Submitted			
897264	predictions.zip	2026-08-22 17:26	Submitted			

AI usage log: Used Gemini Pro for understanding parts of the assignment that were confusing, debugging, and for implementing the cache system in the original version and the modifications needed to submit for the Coda Bench version. Visual Studio Code also did some auto-filling in certain spots. All code used was crossed checked before implementing. Link to the conversations is below:

<https://share.gemini.google/wUbYHpdNGFLg>

<https://share.gemini.google/VxrThKlgTzlF>

Anti-Gaming: To avoid any leakage between the train, test, and validation datasets, no information from later in the session or from validation/test time is used to build the query. This is enforced structurally by the temporal train/val/test split (Q1), and `assert_no_future_leakage()` runs as part of the pipeline build to assert `train.max(timestamp) <= val.min(timestamp) <= test.min(timestamp)`, raising an assertion error if violated. All reported metrics (Q2–Q4) are computed using only features available at serving time i.e., click history, article text/embeddings, and popularity computed strictly from the training window, with no test-time leakage into the candidate scoring or popularity statistics.